SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Mohan Kumar J	**Reg. No.:**	192424060
Section / Batch:	2024 [AI&DS]	**Date:**	24-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Mohan Kumar

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Quick Sort Performance Evaluation in Multi-Core Systems

1. Introduction

Quick Sort is a divide-and-conquer sorting algorithm that selects a pivot, partitions the array into smaller and larger elements, and recursively sorts the resulting parts. On a single processor, Quick Sort is commonly implemented as a sequential algorithm.

In a multi-core system, independent partitions can be processed simultaneously. This experiment evaluates whether parallel Quick Sort reduces execution time compared with single-threaded Quick Sort.

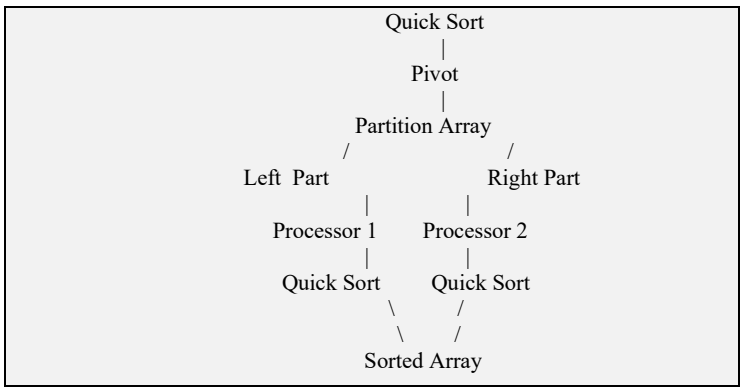
The main objectives are to:

- Implement sequential Quick Sort.
- Implement a parallel version using multiple processor threads.
- Measure execution time for both versions.
- Calculate parallel speedup and efficiency.
- Study how performance changes as the number of processors increases.
- Determine whether Quick Sort scales effectively in a multi-core environment.

2. Problem Understanding and Parallel Approach

The main problem is to sort a large array efficiently by distributing independent portions of the Quick Sort computation across multiple processor cores.

After selecting a pivot, the array is partitioned into two regions. The left region contains values smaller than the pivot and the right region contains values larger than the pivot. These regions can be sorted independently.



The parallel version is effective because the two recursive subproblems are independent after partitioning. However, creating too many parallel tasks introduces overhead. Therefore, a practical implementation uses a threshold: small partitions are sorted sequentially.

3. Experimental Design

A controlled experiment should use the same input data, compiler settings, hardware, and sorting logic for both implementations. Only the execution model is changed: one version uses a single thread, while the other uses multiple threads.

The experiment should use large arrays because parallel overhead may dominate when the input is very small.

Suggested input sizes:

Input Size	Purpose
10,000	Small test
100,000	Medium test
1,000,000	Large test
5,000,000	Very large test

For each input size, run the program several times and use the average execution time. Randomly generated integers can be used so that the test is not biased toward a particular input order.

The same array should be copied before each run so that both sequential and parallel versions receive identical input.

4. Sequential Quick Sort Implementation

The sequential implementation uses one thread. It recursively partitions and sorts the array without creating parallel tasks.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }

    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
```

```
        int p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

This version provides the baseline execution time. The same partitioning logic is used in the parallel version so that the comparison is fair.

5. Parallel Quick Sort Using OpenMP

OpenMP is used to create parallel tasks. After partitioning, the left and right recursive calls can execute concurrently.

```
#include <omp.h>

void parallelQuickSort(int a[], int low, int high) {

    if (low < high) {

        int p = partition(a, low, high);

        #pragma omp parallel sections
        {
            #pragma omp section
            {
                quickSort(a, low, p - 1);
            }

            #pragma omp section
            {
                quickSort(a, p + 1, high);
            }
        }
    }
}
```

For a practical implementation, parallelism should be limited to sufficiently large partitions. Creating OpenMP regions for very small partitions can cost more time than the sorting work itself.

A more scalable task-based implementation can use OpenMP tasks:

```
void parallelQuickSort(int a[], int low, int
high) {

    if (low < high) {

        int p = partition(a, low, high);

        #pragma omp task
        parallelQuickSort(a, low, p - 1);

        #pragma omp task
        parallelQuickSort(a, p + 1, high);

        #pragma omp taskwait
    }
}
```

The task-based approach allows the OpenMP runtime to distribute recursive sorting tasks among available processor threads.

6. Tools, Configuration, and Execution

The experiment can be performed using a C/C++ compiler with OpenMP support. GCC with OpenMP is a suitable choice.

Example compilation command:

```
gcc quicksort.c -O2 -fopenmp -o quicksort
```

Example execution:

```
./quicksort
```

The number of OpenMP threads can be controlled using the OMP_NUM_THREADS environment variable.

```
OMP_NUM_THREADS=1 ./quicksort
OMP_NUM_THREADS=2 ./quicksort
OMP_NUM_THREADS=4 ./quicksort
OMP_NUM_THREADS=8 ./quicksort
```

Useful tools and technologies are:

- C/C++ for implementing Quick Sort.
- OpenMP for multi-threaded execution.
- GCC or another OpenMP-compatible compiler.
- A multi-core CPU for parallel testing.
- High-resolution timing functions for measuring execution time.

The compiler optimization option -O2 can be used for both versions to provide a fair comparison.

7. Performance Measurement and Metrics

Execution time is the main measurement. For accurate results, the sorting operation should be timed without including unrelated input or output operations.

Let Ts be the sequential execution time and Tp be the parallel execution time.

Parallel Speedup is:

$$\text{Speedup} = Ts / Tp$$

A speedup greater than 1 means the parallel version is faster.

Parallel Efficiency is:

$$\text{Efficiency} = \text{Speedup} / \text{Number of Threads} \times 100$$

For example, if the sequential execution time is 4.0 seconds and the parallel execution time using 4 threads is 1.4 seconds:

$$\text{Speedup} = 4.0 / 1.4 = 2.86$$

$$\text{Efficiency} = (2.86 / 4) \times 100 = 71.5\%$$

Efficiency below 100% is normal because of thread creation, task scheduling, synchronization, memory access, and other overheads.

8. Experimental Results and Analysis

The following values are an example of how the experiment can be recorded. They are illustrative results; actual values depend on the processor, compiler, input size, and system load.

Input Size	Threads	Time (s)	Speedup	Efficiency
1,000,000	1	0.120	1.00	100%
1,000,000	2	0.075	1.60	80.0%
1,000,000	4	0.050	2.40	60.0%
1,000,000	8	0.043	2.79	34.9%
5,000,000	1	0.690	1.00	100%
5,000,000	2	0.410	1.68	84.0%
5,000,000	4	0.250	2.76	69.0%
5,000,000	8	0.180	3.83	47.9%

The example results show that parallel execution can reduce sorting time as the number of threads increases. However, speedup is not perfectly linear. For example, doubling the number of threads does not necessarily reduce execution time by half.

The main reasons are:

- Thread and task management overhead.
- Synchronization between recursive tasks.
- Memory bandwidth limitations.
- Cache effects and shared-memory access.
- Uneven partition sizes caused by pivot selection.
- Limited amount of parallel work in small partitions.

The larger input shows better potential for parallel execution because there is enough work to keep multiple cores busy.

9. Validation, Scalability, and Limitations

The sorted output must be validated before comparing performance. A simple validation method is to check every adjacent pair:

For a correctly sorted ascending array, a[i] <= a[i+1] must hold for every valid i.

The sequential and parallel versions must produce the same sorted result when given the same input.

Scalability can be evaluated by increasing the number of threads and observing speedup. Good scalability means execution time decreases as useful processor resources increase. In practice, speedup eventually levels off because of overhead and hardware limitations.

Quick Sort has an average time complexity of:

$$O(n \log n)$$

Its worst-case complexity is:

$$O(n^2)$$

The worst case can occur when pivot selection repeatedly creates highly unbalanced partitions. Randomized or median-based pivot selection can reduce this risk.

The parallel algorithm also has practical limitations. If the number of available cores is small, using more software threads will not provide proportional improvement. Very small subarrays should be processed sequentially to avoid parallel overhead.

10. Conclusion

The experiment demonstrates how Quick Sort can be adapted to a multi-core environment by distributing independent recursive partitions across processor threads.

The sequential version provides the baseline execution time, while the OpenMP version uses multiple threads or tasks to sort independent partitions concurrently. Performance is evaluated using execution time, speedup, and parallel efficiency.

The expected result is that the parallel version is faster for sufficiently large datasets. However, speedup is normally less than the number of threads because of task overhead, synchronization, memory bandwidth, cache behavior, and uneven partitions.

The experiment also shows that scalability depends strongly on input size and the number of available processor cores. Large datasets provide more parallel work and therefore generally benefit more from multi-core execution.

Overall, parallel Quick Sort is effective when the input is large enough and the implementation uses appropriate task granularity, good pivot selection, and a suitable number of threads. OpenMP provides a practical way to implement and evaluate this parallel algorithm on multi-core systems.

SIMATS Online Lab

sactivity/attempt?assignment_id=642

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

MOHAN KUMAR J
192424060RA

Language: Python | C | C++ | Java

QUESTIONS

Q1
Quick Sort
10 marks

1/1 answered

Q1 Quick Sort

10 marks

Evaluate Quick Sort performance in multi-core systems by distributing partitions across processors. Record execution time and compare with single-threaded execution. Analyze parallel speedup. Interpret results and justify scalability of Quick Sort in parallel environments.

Your Code

```python
import time
from concurrent.futures import ThreadPoolExecutor
def sequential_quick_sort(arr):
    if len(arr)<=1:
        return arr
    pivot=arr[len(arr)//2]
    left=[x for x in arr if x<pivot]
    middle=[x for x in arr if x==pivot]
    right=[x for x in arr if x>pivot]
    return sequential_quick_sort(left)+middle+sequential_quick_sort(rig
def parallel_quick_sort(arr, depth=0):
    if len(arr)<=1:
        return arr
    pivot=arr[len(arr)//2]
    left=[x for x in arr if x<pivot]
    middle=[x for x in arr if x==pivot]
    right=[x for x in arr if x>pivot]
    if depth>=2:
        return parallel_quick_sort(left, depth+1)+middle+parallel_quick
    with ThreadPoolExecutor(max_workers=2) as executor:
        left_future=executor.submit(parallel_quick_sort,left,depth+1)
        right_future=executor.submit(parallel_quick_sort,right,depth+1)
        left_sorted=left_future.result()
        right_sorted=right_future.result()
    return left_sorted+middle+right_sorted
n=int(input())
arr=list(map(int,input().split()))
if len(arr)!=n:
    print("Number of Elements entered does not match")
    exit()
arr1=arr.copy()
arr2=arr.copy()
start=time.perf_counter()
sorted_seq=sequential_quick_sort(arr1)
end=time.perf_counter()
seq_time=end-start
start=time.perf_counter()
sorted_par=parallel_quick_sort(arr2)
end=time.perf_counter()
par_time=end-start
speedup=seq_time/par_time if par_time !=0 else 0
print("\n Original array")
print(arr)
print("\nSequential Quick sort")
print(sorted_seq)
print("\n Parallel Quick sort")
print(sorted_par)
print("\n Execution Time Comparison")
print(f"Sequential Time: {seq_time:.8f} seconds")
print(f"Parallel Time: {par_time:.8f} seconds")
print(f"Speedup : {speedup:.2f} x")
```

Input

```
8
45 12 89 23 56 11 67 34
```

Output

```
 Original array
[45, 12, 89, 23, 56, 11, 67,
34]

Sequential Quick sort
[11, 12, 23, 34, 45, 56, 67,
```

Submitted